

Lab-9 Hash

1. Task-1: Study various Hash algorithms and identify various platform where they are using.

What is a Cryptographic Hash Function?

- A mathematical algorithm that takes an input of any size—such as text, files, or data streams—and produces a fixed-length output, known as a **Hash value, message digest, or hash.**
- Common Hash Algorithms & Platforms Where They're Used:

Algorithm	Output Size	Status	Where Used
MD5	128-bit	Broken (collisions)	Old file checksums, legacy systems (NOT recommended for security)
SHA-1	160-bit	Broken (collisions)	Old TLS certificates, Git object IDs (being migrated), legacy protocols
SHA-256 (SHA-2 family)	256-bit	Secure (as of now)	TLS/SSL, digital signatures, Bitcoin blockchain, file integrity, JWT signing, code signing
SHA-384 / SHA-512 (SHA-2)	384 / 512-bit	Secure	High-security digital signatures, PKI, certificates, secure protocols
SHA-3 (Keccak)	224/256/384/512	Secure	Newer standards, embedded / hardware implementations, some blockchains (e.g. Keccak-256)

2. Task-2: Implement a basic digital signature system using a hash function and asymmetric cryptography (e.g., RSA). Sign a message with a private key and verify the signature using the corresponding public key.

- I have implemented basic digital signature system using a hash function and asymmetric cryptography by using a python code that uses 2 prime number, public exponent for the key generation and then finally the Digital signatures generation.

• Python code is:

```
import hashlib
def gcd(a, b):
    while b != 0:
        a, b = b, a % b
    return a
```

```
def mod_inverse(e, phi):
    # Extended Euclid's Algorithm to find d such that (d * e) % phi
    == 1
    old_r, r = e, phi
    old_s, s = 1, 0

    while r != 0:
        q = old_r // r
        old_r, r = r, old_r - q * r
        old_s, s = s, old_s - q * s

    if old_s < 0:
        old_s += phi
    return old_s
```

```
def sha256_hash(message: str) -> int:
    """
    Hash the message with SHA-256 and return as an integer.
    """
    digest = hashlib.sha256(message.encode('utf-8')).hexdigest()
    return int(digest, 16) # convert hex string to integer
```

----- Key Generation (RSA) -----

```
print("=== RSA Key Generation ===")
p = int(input("Enter prime number p: "))
q = int(input("Enter prime number q: "))
n = p * q
phi = (p - 1) * (q - 1)
print(f"\nComputed n = p * q = {n}")
print(f"Computed phi(n) = (p-1)*(q-1) = {phi}")
while True:
    e = int(input("\nEnter public exponent e (1 < e < phi and
gcd(e,phi)=1): "))
    if 1 < e < phi and gcd(e, phi) == 1:
        break
    else:
        print("Invalid e. It must be 1 < e < phi and gcd(e, phi) == 1.
Try again.")
d = mod_inverse(e, phi)
print("\nPublic Key (e, n) =", (e, n))
print("Private Key (d, n) =", (d, n))
print("\n=== Digital Signature Generation ===")
message = input("Enter the message to sign: ")
```

```
# Step 1: Hash the message
h = sha256_hash(message)
```

```
print("\nSHA-256 hash of message (as integer):")
print(h)
```

```
# Step 2: Create signature:  $s = h^d \bmod n$ 
signature = pow(h, d, n)
print("\nDigital Signature (integer value):")
print(signature)
print("\n=== Signature Verification ===")
message_to_verify = input("Enter the message to verify: ")
```

```
# Recompute hash of the message
h_verify = sha256_hash(message_to_verify)
print("\nRecomputed SHA-256 hash of message (as integer):")
print(h_verify)
```

```
# Decrypt signature using public key:  $h' = s^e \bmod n$ 
h_from_signature = pow(signature, e, n)
print("\nHash recovered from signature using public key:")
print(h_from_signature)
```

```
# Compare hashes
if h_verify == h_from_signature:
    print("\n✅ Signature is VALID: message is authentic and not
modified.")
else:
    print("\n❌ Signature is INVALID: message has been changed
or signature is wrong.")
```

Output: === RSA Key Generation ===

Enter prime number p: 3

Enter prime number q: 7

Computed $n = p * q = 21$

Computed $\phi(n) = (p-1)*(q-1) = 12$

Enter public exponent e ($1 < e < \phi$ and $\gcd(e,\phi)=1$):

7

Public Key (e, n) = (7, 21)

Private Key (d, n) = (7, 21)

=== Digital Signature Generation ===

Enter the message to sign: encrypt

SHA-256 hash of message (as integer):

895705427487623765316694744646671784317481449

87730049609001327180736215261406

Digital Signature (integer value):

2

=== Signature Verification ===

Enter the message to verify: encrypt

Recomputed SHA-256 hash of message (as integer):

630771521698741977202756501549235176821613211

85708578260190643574431838384022

Hash recovered from signature using public key:

2

✗ Signature is INVALID: message has been changed or signature is wrong.